

SmallMartApp: Smart Mart POS & Inventory Management System

A Desktop Point of Sale and Inventory Architecture Built with .NET 8 WPF, Clean Architecture, MVVM & KHQR Payments

**Seng Porkeat¹, Sim Pengseang¹, Cheng Devith¹, Tourn Vuthy¹, Toch Ratana¹, Va Eric¹,
Sorn Sophamarinet¹, Korm Taingan¹, Leng Senghong¹, Mao Piseth¹, Kay Keo¹, Srorn Sokcheat¹**

¹Institute of Science and Technology Advanced Development (ISTAD) & Royal University of Phnom Penh (RUPP)

Faculty of Science, Department of Computer Science — Object-Oriented Analysis & Design (OOAD)

#40, St. 273, Sangkat Boeung Kak I, Khan Toul Kork, Phnom Penh, Cambodia

Email: {pengseangsim210, alexkgm2412, chengdevith5, vuthytoun168, ericva014, ratanatoch68, netsorn7777, kormtaingan99, lengsenghong168, pisethmao2002, keokay888, sokcheatsrornr@gmail.com}

Abstract — SmallMartApp is a production-grade, modular Windows desktop Point of Sale (POS) and inventory management system engineered specifically for convenience stores, mini-marts, and retail kiosks. Built upon Microsoft .NET 8 and Windows Presentation Foundation (WPF), the platform strictly implements Object-Oriented Analysis and Design (OOAD) principles, Clean Architecture (Vertical Slice feature separation), and the Model-View-ViewModel (MVVM) pattern backed by the CommunityToolkit.Mvvm framework. SmallMartApp provides an end-to-end retail solution spanning real-time barcode scanning, cashier shift cash-drawer auditing, customer loyalty reward accounting, automated thermal receipt generation, and a fully compliant National Bank of Cambodia (NBC) Bakong KHQR dynamic QR payment engine. Persistence is abstracted via Microsoft Entity Framework Core 8 with dual support for enterprise Microsoft SQL Server and embedded SQLite. Decoupled device abstractions allow hardware peripherals (barcode scanners, thermal receipt printers) to be seamlessly swapped between physical hardware and simulated environments, ensuring maximum testability, operational stability, and low maintenance overhead.

1. Introduction

In today's fast-paced retail ecosystem, efficient point-of-sale operations and rigorous inventory tracking are fundamental prerequisites for operational sustainability and customer satisfaction. While large supermarket chains possess the financial capital to deploy expensive ERP solutions, small-to-medium retail enterprises (SMEs), convenience stores, and neighborhood marts frequently grapple with fragmented, manual processes. Store operators typically rely on disjointed spreadsheets, standalone electronic cash registers, or paper logbooks. These outdated methods introduce severe operational vulnerabilities, including inventory shrinkage, stock-out emergencies, cash register reconciliation discrepancies, and lengthy customer queues at checkout.

Furthermore, the rapid evolution of digital finance across Southeast Asia—specifically the widespread adoption of Cambodia's national payment gateway (Bakong) and the standardized KHQR code system—has fundamentally shifted consumer expectations toward frictionless cashless payments. Traditional cash registers fail to bridge physical cash transactions with dynamic digital QR payments, leaving merchants to manually reconcile separate bank payment notifications against physical receipts.

SmallMartApp was conceptualized, designed, and engineered to resolve these challenges comprehensively. By leveraging modern C# 12 and the .NET 8 desktop runtime, SmallMartApp delivers a high-performance, robust, and

visually intuitive retail management system. It unifies frontline cashier point-of-sale checkout, automated barcode acquisition, customer loyalty incentives, cashier shift float management, supplier purchase orders, and multi-currency KHQR payment processing into a single, cohesive desktop application. By adhering strictly to Clean Architecture and MVVM design principles, the software decouples user interface interactions from business domain logic and database persistence, ensuring enterprise-grade maintainability, testability, and extensibility.

2. Background & Problem Domain

SmallMartApp was developed as an advanced Object-Oriented Analysis & Design (OOAD) enterprise project by IT researchers and software engineering students at the Institute of Science and Technology Advanced Development (ISTAD) in collaboration with the Royal University of Phnom Penh (RUPP). The project's mission is to provide an accessible, rock-solid, and extensible desktop platform that eliminates retail operational friction for small marts, supermarkets, and grocery kiosks.

Through empirical study of local convenience mart operations in Phnom Penh, four critical systemic problems were identified in existing retail workflows:

- **1. Inventory Shrinkage and Stock Discrepancies:** Without automated real-time inventory deductions upon barcode scan, inventory numbers diverge from physical shelf stock. Merchants frequently experience stock-outs of popular items or over-order perishable goods without automated low-stock warnings.
- **2. Cashier Shift Auditing and Financial Gaps:** In multi-shift operations (morning, afternoon, night), untracked cash floats lead to untraceable cash shortages. Without structured shift opening floats, system-expected cash balances, and end-of-shift cash drawer reconciliations, store owners cannot identify theft or accounting errors.
- **3. Checkout Delays and Fragmented Payments:** Cashiers are forced to manually enter item amounts or switch between separate bank terminal apps to accept digital payments. Generating dynamic QR codes with exact transaction amounts and validating cash-change calculations within seconds is essential.
- **4. Lack of Customer Loyalty Retention:** Small marts struggle to maintain customer loyalty programs without complex hardware. An integrated phone-lookup loyalty system that automatically accrues points per dollar spent and permits seamless point redemption at checkout provides substantial competitive advantage.

SmallMartApp Engineering Philosophy

The core architectural philosophy of SmallMartApp is 'Simplicity on the Surface, Rigor in the Core'. Frontline cashiers benefit from a fast, touch-friendly, keyboard-accelerated POS interface, while store managers benefit from transparent shift auditing, stock threshold alerts, and rock-solid relational persistence.

3. System Architecture

SmallMartApp is architected following Clean Architecture, Vertical Slice Feature Modularization, and the Model-View-ViewModel (MVVM) presentation pattern. The solution is structured into three distinct physical layers enforcing strict inward dependency flow, accompanied by an independent automated testing suite:

- **SmallMartApp.Core (The Domain Brain):** Pure C# class library containing zero external framework dependencies. Encapsulates domain entity models, business enumerations, service contract interfaces (IProductService, ISalesService, IKhqrService, etc.), hardware driver abstractions (IBarcodeScanner, IReceiptPrinter), and the generic Result<T> pattern for deterministic error handling.
- **SmallMartApp.Infrastructure (Data & Peripherals):** Implements the persistence and hardware contracts defined in Core. Manages Microsoft Entity Framework Core 8 DbContext, SQL Server and SQLite database providers,

relational entity configurations, database seeding, physical/mock hardware drivers, and the EMVCo KHQR byte generation engine.

- **SmallMartApp.UI (WPF Presentation Layer):** Windows Presentation Foundation desktop application containing Views (XAML layout), ViewModels (CommunityToolkit.Mvvm state handlers), ValueConverters, UI notification services, thermal receipt preview windows, and the DI container bootstrapper (App.xaml.cs).
- **SmallMartApp.Tests (Automated Verification):** xUnit testing project utilizing in-memory database providers to verify sales processing, stock decrements, cash calculations, and KHQR checksum integrity in continuous integration pipelines.

3.1 Selected Technologies

The following table categorizes the primary technological stack, software tools, frameworks, and packages chosen for SmallMartApp:

Component / Category	Selected Technology	Description & Architectural Role
Programming Language	C# 12 (.NET 8.0 SDK)	Modern, strongly typed language featuring records, pattern matching, nullable references, and high performance.
UI Presentation Engine	Windows Presentation Foundation (WPF)	Hardware-accelerated desktop UI engine with rich XAML declarative markup, DataBinding, and DataTemplates.
MVVM Framework	CommunityToolkit.Mvvm (v8.3.2)	Roslyn source-generator driven framework eliminating boilerplate via [ObservableProperty] and [RelayCommand].
Data Persistence (ORM)	Entity Framework Core 8.0.11	High-performance object-relational mapper managing database schema, LINQ queries, relationships, and seed data.
Relational Databases	Microsoft SQL Server & SQLite	SQL Server Express for enterprise multi-terminal LAN setups; SQLite for standalone zero-configuration local deployments.
Dependency Injection	Microsoft.Extensions.Hosting & DI	Inversion-of-Control container registering transient business services, singleton mock devices, and view models.
Digital Payment Engine	QRCoder (v1.8.0) & Custom KHQR	Generates high-resolution dynamic QR codes compliant with National Bank of Cambodia (NBC) EMVCo standards.
Hardware Abstraction	IBarcodeScanner & IReceiptPrinter	Decoupled hardware interfaces allowing seamless switching between physical USB POS devices and mock testing drivers.
Unit & Integration Testing	xUnit & InMemory EF Core	Automated test suite verifying checkout rules, cash change mathematics, and stock deduction constraints.
Development Environments	Visual Studio 2022 / JetBrains Rider	Primary IDEs utilized for compilation, XAML visual design, debugging, and solution maintenance.

3.2 Selected Architectural Patterns & Design Principles

The architecture of SmallMartApp is governed by strict object-oriented design patterns ensuring separation of concerns, loose coupling, and testability:

A. The MVVM (Model-View-ViewModel) Triad:

In contrast to traditional Windows Forms where business logic, SQL commands, and UI control manipulations are mixed within code-behind files, SmallMartApp strictly enforces the MVVM architectural separation:

Layer	UI Interactivity	Core Contents & Responsibilities	Project Implementation Example
Model	Zero UI Knowledge	Pure POCO classes representing database entities and domain rules. No references to WPF controls or buttons.	Product.cs, Sale.cs, CashierShift.cs
View	Visual Layout Only	Declarative XAML definitions specifying margins, colors, controls, data templates, and bindings. Zero business logic.	PosCheckoutView.xaml, DashboardView.xaml
ViewModel	100% UI Logic	Maintains UI state, handles button commands via [RelayCommand], executes validations, and coordinates service calls.	PosCheckoutViewModel.cs, DashboardViewModel.cs

B. Modern Two-Way Data Binding and Relay Commands:

Rather than accessing UI controls by name (e.g., txtBarcode.Text), SmallMartApp utilizes Roslyn source generators from CommunityToolkit.Mvvm. Backing fields decorated with [ObservableProperty] automatically synthesize INotifyPropertyChanged events, allowing XAML controls to update instantly upon property modification. Methods decorated with [RelayCommand] automatically generate asynchronous ICommand properties with built-in execution state tracking and button disablement.

```
// Exemplary ViewModel Implementation in SmallMartApp
[ObservableProperty]
private string _barcodeInput = string.Empty;

[ObservableProperty]
private ObservableCollection<CartItemViewModel> _cartItems = new();

[RelayCommand(CanExecute = nameof(CanCompleteCheckout))]
private async Task CompleteCheckoutAsync()
{
    var result = await _salesService.ProcessSaleAsync(cartData, CashReceived, CurrentShiftId);
    if (result.IsSuccess) { StatusMessage = "Sale completed!"; ResetCart(); }
}
```

C. Decoupled View Navigation via DataTemplates:

SmallMartApp eliminates hard-coded navigation logic between screens. The main application window hosts a single ContentControl bound to MainViewModel.CurrentView. In App.xaml, global DataTemplates map ViewModel types directly to View UserControls:

```
<DataTemplate DataType="{x:Type vm:PosCheckoutViewModel}">
    <views:PosCheckoutView />
</DataTemplate>
```

When the user clicks a navigation item (e.g., POS, Inventory, Shift, Customers), the MainViewModel simply sets

CurrentView to the appropriate ViewModel instance. WPF automatically resolves the visual blueprint, keeping ViewModels 100% free of UI references.

3.3 Entity Relationship Diagram (ERD) & Database Design

SmallMartApp's relational database schema is modeled to ensure strict referential integrity, normalized relations, and high query performance. The schema consists of 10 primary relational tables configured using EF Core Fluent API:

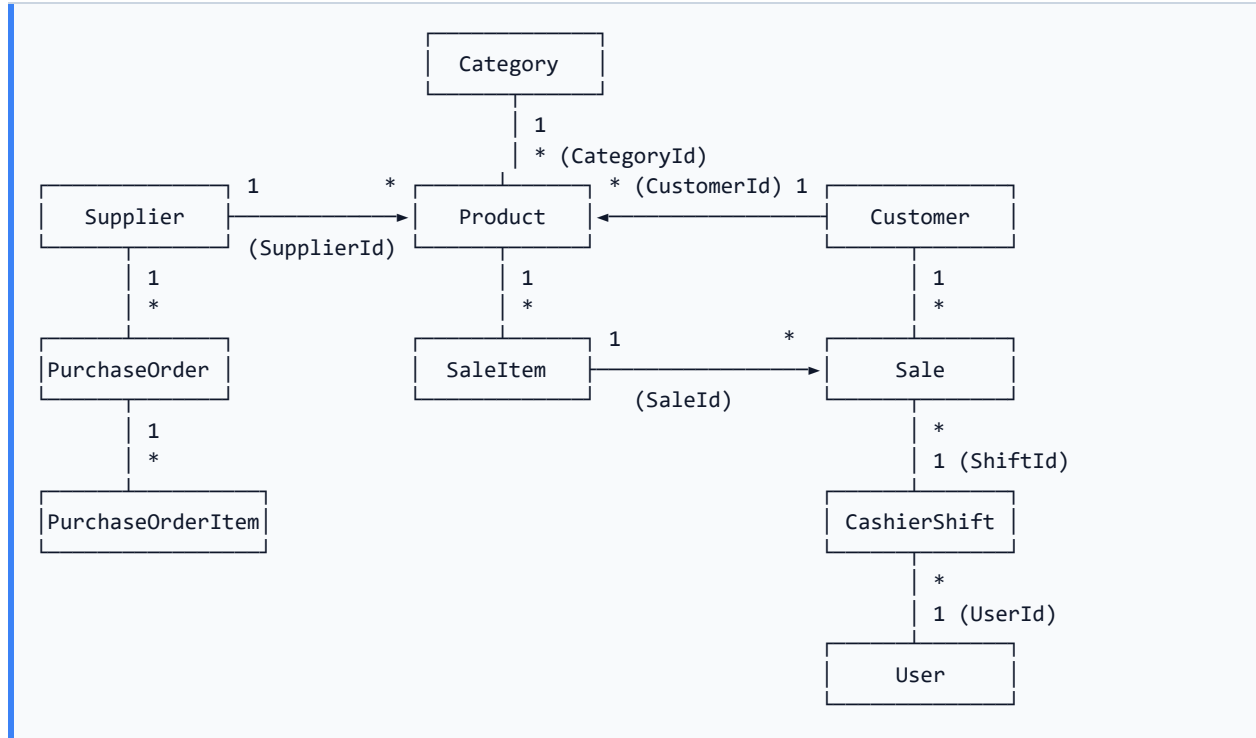


Table Name	Primary Key	Foreign Keys	Key Attributes & Constraints
Categories	Id (int, PK)	None	Name (VARCHAR 100, Required), Description (VARCHAR 250), CreatedAt
Suppliers	Id (int, PK)	None	CompanyName (VARCHAR 150), PhoneNumber (VARCHAR 30), ContactPerson, Address
Products	Id (int, PK)	CategoryId (FK) SupplierId (FK)	Barcode (VARCHAR 50, UNIQUE Index), Name (VARCHAR 150), CostPrice (18,2), SellPrice (18,2), StockQuantity (int), LowStockAlertThreshold (int), IsActive (bool)
Customers	Id (int, PK)	None	PhoneNumber (VARCHAR 30, UNIQUE Index), FullName (VARCHAR 120), Points (int, Loyalty Accrual)
Users	Id (int, PK)	None	Username (VARCHAR 50, UNIQUE Index), FullName, WorkingShift, PasswordHash, Role (Admin, Cashier), IsActive
CashierShifts	Id (int, PK)	UserId (FK)	StartingCash (18,2), ExpectedCash (18,2), ActualCash (18,2), Status (Open, Closed), StartedAt, EndedAt
Sales	Id (int, PK)	ShiftId (FK) CustomerId (FK)	ReceiptNumber (VARCHAR 50, Unique), TotalAmount (18,2), DiscountAmount (18,2), CashReceived (18,2), ChangeGiven (18,2), PaymentMethod (Cash, KHQR, Card), CreatedAt

Table Name	Primary Key	Foreign Keys	Key Attributes & Constraints
SaleItems	Id (int, PK)	SaleId (FK, Cascade) ProductId (FK)	ProductName (VARCHAR 150), UnitPrice (18,2), Quantity (int)
PurchaseOrders	Id (int, PK)	SupplierId (FK)	TotalCost (18,2), Status (Pending, Completed), OrderDate (DateTime)
PurchaseOrderItems	Id (int, PK)	PurchaseOrderId (FK, Cascade) ProductId (FK)	ProductName (VARCHAR 150), UnitCost (18,2), Quantity (int)

3.4 Core Subsystems & Business Workflow Details

A. Point of Sale (POS) Checkout & Transaction Flow:

The checkout process coordinates the interaction between physical barcode input, cart item accumulation, discount and loyalty calculations, stock deduction, and receipt printing. When a sale is confirmed, the SalesService executes within an atomic database transaction: validating item stock availability, decrementing physical inventory, computing change given, accruing customer loyalty points (1 point per dollar spent), logging the transaction to the active cashier shift, and dispatching formatted thermal receipt data to the printer driver.

B. Cashier Shift & Cash Float Reconciliation:

To prevent cash leakage and ensure accountability, cashiers must initiate an official shift by entering an opening cash float (StartingCash). Throughout the shift, all cash checkout transactions automatically increment the ExpectedCash balance. Upon shift termination, the cashier physically counts the cash drawer and submits ActualCash. The system calculates variance: $\text{Discrepancy} = \text{ActualCash} - \text{ExpectedCash}$. This audit trail is permanently bound to the cashier user record.

C. Cambodian National Bank (NBC) KHQR Payment Engine:

SmallMartApp integrates a custom KHQR generator compliant with the EMVCo standard mandated by the National Bank of Cambodia (NBC). The engine dynamically formats Tag-Length-Value (TLV) payloads supporting both standard NBC Bakong accounts (e.g. user@aba) and ACLEDA Bank EMVCo routing configurations (Tag 29 sub-tags 00, 01, 02 with Tag 39 dual currency 2CCY indicators). Payloads are validated with a 16-bit CRC-CCITT (polynomial 0x1021, initial 0xFFFF) checksum and rendered directly into QR Code PNG images on the cashier screen.

4. Evaluation

SmallMartApp was evaluated through rigorous simulated retail operations, multi-item checkout stress tests, and automated unit test suites. The evaluation demonstrates significant operational advantages alongside distinct technical trade-offs.

4.1 Project Strengths

- Architectural Rigor & Clean Separation:** Strict adherence to Clean Architecture and MVVM guarantees that business rules (SalesService, Stock deductions) are completely independent of presentation controls. The solution can be refactored, extended, or migrated to web/mobile APIs without rewriting domain logic.
- Modern Cashless Payment Integration (KHQR):** Native generation of EMVCo-compliant KHQR dynamic codes eliminates human cashier input error when entering transaction amounts, drastically accelerating checkout speed for mobile banking customers across Cambodia.

- **Real-Time Shift Auditing & Cash Accountability:** The shift float tracking engine prevents cashier cash discrepancies, providing store owners with clear, auditable logs of expected versus actual cash amounts per shift.
- **Decoupled Hardware Driver Layer:** By utilizing IBarcodeScanner and IReceiptPrinter interfaces, the software runs seamlessly in mock development mode without physical POS peripherals, while instantly binding to real USB laser scanners and thermal ESC/POS printers in production.
- **Dual Database Persistence Flexibility:** EF Core configuration enables zero-configuration SQLite for single kiosk stores, or Microsoft SQL Server for multi-lane grocery marts.
- **Automated Test Verification:** Comprehensive xUnit test coverage validates critical checkout math, stock decrement thresholds, and KHQR string checksum generation.

4.2 Project Weaknesses & Limitations

- **Windows Operating System Dependency:** Because the user interface is constructed using Windows Presentation Foundation (WPF), SmallMartApp is constrained to Microsoft Windows environments (Windows 10/11). Cross-platform deployment (macOS, Linux, Android) would require a future migration to .NET MAUI or Avalonia.
- **Localized Persistence Without Real-Time Cloud Sync:** The current release operates on local area networks (LAN) or single machines. Multi-branch supermarkets operating across different cities require a centralized cloud synchronization service to aggregate inventory and sales data in real time.
- **Lack of Direct Serial ESC/POS Hardware Drivers:** Receipt printing is currently handled via WPF document drawing and text file logging. Future releases should implement raw ESC/POS binary command transmission via Serial/COM and TCP/IP sockets for ultra-fast thermal roll cutting and cash drawer ejection.
- **Reporting Export Formats:** While real-time dashboard KPIs are displayed on screen, the system currently lacks built-in export modules to generate downloadable Excel (.xlsx) or PDF financial statements for store accounting.

5. Conclusion & Future Work

SmallMartApp demonstrates the successful application of modern Object-Oriented Analysis & Design (OOAD) principles, Clean Architecture, and WPF MVVM patterns to real-world retail automation. By addressing the critical pain points of inventory shrinkage, cash drawer discrepancies, and fragmented digital payment methods, SmallMartApp provides small-to-medium retail stores with an enterprise-caliber point of sale and inventory management solution. The integration of Cambodia's national KHQR standard bridges traditional physical retail with modern cashless convenience, offering a streamlined experience for cashiers, customers, and store managers alike.

Future development milestones for SmallMartApp include:

1. Cloud API Gateway & Synchronization: Implementing an ASP.NET Core Web API backend with Azure SQL to facilitate multi-branch inventory transfers and centralized reporting.
2. Cross-Platform Mobile Management App: Building a companion mobile application using .NET MAUI allowing store owners to view live sales KPIs, low-stock notifications, and shift closures from smartphones.
3. Predictive Restocking & Analytics: Incorporating machine learning demand forecasting to predict stock reorder dates based on historical sales trends.

References

- [1] Microsoft Corporation. (2024). Windows Presentation Foundation (WPF) .NET 8 Documentation.
<https://learn.microsoft.com/en-us/dotnet/desktop/wpf/>
- [2] Microsoft Community Toolkit. (2024). MVVM Toolkit Introduction & Architecture Guidelines.
<https://learn.microsoft.com/en-us/dotnet/communitytoolkit/mvvm/>

- [3] Microsoft Corporation. (2024). Entity Framework Core 8 Overview & Best Practices. <https://learn.microsoft.com/en-us/ef/core/>
- [4] National Bank of Cambodia (NBC). (2022). KHQR Technical Specification & Bakong EMVCo Standard. Phnom Penh, Cambodia.
- [5] Martin, R. C. (2017). Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall.
- [6] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.
- [7] EMVCo, LLC. (2020). EMV QR Code Specification for Payment Systems (EMVCo QRCPS). <https://www.emvco.com/>